SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Code of Conduct:

I N.A.Sriharan / 1911260036 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

ANSWER:

1. How Greedy Best-First Search behaves

Greedy Best-First Search selects the node that appears closest to the destination based only on the heuristic value \(h(n)\), such as straight-line distance.

- Strengths:

- Makes decisions quickly because it focuses on the estimated distance to the destination.
- Can find a route quickly when the environment is relatively simple.
- Useful when emergency medicines must be delivered urgently.

- Weaknesses:

- Ignores the actual cost already incurred.
- May choose a route through blocked roads or areas with dangerous weather.
- Does not guarantee the shortest or safest route.

Example: The drone may choose a route that looks shortest but has strong winds or blocked paths, causing delays and increasing risk.

2. How A* Search handles the scenario

A* Search combines the actual cost of reaching a node with the estimated remaining cost to the destination.

The evaluation function is:

$$ f(n)=g(n)+h(n) $$

Where:

- $g(n)$ = Actual cost from the starting point to the current node, including travel time and other modeled costs.
- $h(n)$ = Estimated cost from the current node to the destination.
- $f(n)$ = Estimated total cost of the route through that node.

Strengths:

- Considers both the cost already incurred and the estimated remaining cost.
- Can find an optimal route when the cost model and heuristic satisfy the required conditions.
- Can account for weather, terrain, and travel delays if these are included in the cost function.

Weaknesses:

- May require more time and memory than Greedy Best-First Search.
- May need to recalculate routes when obstacles or weather conditions change.
- Cannot guarantee the best real-world route if its cost estimates are inaccurate or the environment changes unexpectedly.

Example: Even if one route is shorter, A* may select a slightly longer route if it has lower estimated travel time and risk.

3. Impact of heuristic accuracy on both algorithms

A heuristic estimates the remaining cost to reach the destination. Its accuracy affects how efficiently both algorithms search.

Factor	Greedy Best-First Search	A* Search
Accurate heuristic	Usually guides the search toward the destination quickly.	Helps find an efficient route with fewer unnecessary expansions.
Inaccurate heuristic	May choose a poor route because it relies heavily on the heuristic.	May explore unnecessary nodes or lose optimality if the heuristic overestimates.
Straight-line distance	Useful for estimating geographical distance.	Useful, provided it is compatible with the actual cost function.

Weather and terrain	Does not consider their effects unless incorporated into the heuristic.	Can account for them through the path cost and, where appropriate, the heuristic.

An admissible heuristic never overestimates the minimum remaining cost. This property helps A* guarantee an optimal solution under the standard search assumptions.

4. Impact of dynamic changes and blocked paths

The environment may change while the drone is travelling. For example, a previously safe route may become inaccessible due to flooding or worsening weather.

The effects include:

- Route failure: The drone may attempt to follow a route that is no longer available.
- Replanning: The search algorithm must update its map and calculate a new route.
- Additional delay: Repeated route calculations can consume time and computational resources.
- Safety risks: Outdated information may direct the drone toward dangerous areas.

Solution: The drone should receive real-time updates from sensors and weather systems, remove blocked paths from the available routes, update movement costs, and replan when necessary. Incremental search algorithms can also reduce repeated work when the environment changes frequently.

5. Recommended algorithm and justification

Recommendation: A Search with real-time replanning and safety constraints.*

A* is more suitable when the drone must balance travel time, terrain difficulty, weather conditions, and safety instead of simply choosing the route that appears shortest.

Criterion	**Greedy Best-First Search**	**A* Search**
Decision speed	Generally faster	May require more computation
Considers actual path cost	No	Yes
Optimality	Not guaranteed	Guaranteed under suitable conditions
Handles variable travel costs	Limited unless specially adapted	Yes, through the cost function
Safety	Requires additional safety checks	Can incorporate modeled risks and safety constraints

To improve performance, the drone should:

1. Use A* with an admissible heuristic.
2. Include travel time, weather, terrain, and risk in the cost model.
3. Treat dangerous or prohibited routes as unavailable.
4. Replan whenever important real-time information changes.
5. Use a safe fallback route or landing procedure if no safe route is available

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

ANSWER:

1. Formulate the problem as an optimization problem and explain why traditional search is inefficient

The objective is to optimize traffic signal timings across hundreds of intersections to minimize total waiting time, fuel consumption, and traffic congestion.

Optimization problem formulation:

- State: Current traffic conditions and signal timings at all intersections.
- Decision variables: Green-light duration, red-light duration, and signal coordination between intersections.
- Objective function: Minimize total waiting time, fuel consumption, and congestion.

$$\text{Minimize } F(x)=w_1W+w_2F_c+w_3C$$

Where:

- W = Total vehicle waiting time.
- F_c = Total fuel consumption.

- C = Traffic congestion.
- w_1,w_2,w_3 = Weights assigned to each objective.
- x = Set of traffic signal timings.

Why traditional search is inefficient:

Traditional search methods such as BFS and DFS explore many possible states. With hundreds of intersections and numerous signal timings, the search space becomes extremely large. Traffic conditions also change continuously, making it difficult to calculate and maintain an optimal solution using exhaustive search.

2. Explain how Hill Climbing works and identify its limitations

Hill Climbing is a local search algorithm that starts with an initial solution and repeatedly moves to a neighboring solution that improves the objective function.

Working:

1. Start with random or existing traffic signal timings.
2. Calculate total waiting time, fuel consumption, and congestion.
3. Modify the timing of one or more traffic signals.
4. Evaluate the new solution.
5. Accept the better solution and repeat until no improvement is found.

Limitations:

- May get stuck in a local optimum.
- Cannot easily escape plateaus or ridges.
- Does not guarantee a global optimum.
- May struggle when traffic conditions change rapidly.

Example: Improving one intersection may reduce its waiting time but increase congestion at nearby intersections.

3. Explain local maxima, plateaus, and ridges with real-world examples

Problem	**Meaning**	**Traffic example**
Local maximum (or local optimum)	A solution better than its nearby alternatives but not the best overall	Signal timings reduce congestion locally, but a different citywide timing plan would work better.
Plateau	A flat region where neighboring solutions have	Several timing adjustments produce almost the same waiting time, so the algorithm

	similar objective values	cannot identify a better direction.
Ridge	A narrow path of improvement that is difficult to follow through simple local changes	Coordinating signals across several roads requires multiple timing changes together; changing only one signal may not improve traffic.

Note: Because this is a minimization problem, the algorithm is effectively looking for a local minimum in the cost function, even though the common Hill Climbing terminology often refers to local maxima.

4. Explain how Simulated Annealing and Genetic Algorithms overcome these limitations

A. Simulated Annealing (SA)

Simulated Annealing sometimes accepts a worse solution to escape a local optimum. As the algorithm progresses, it gradually reduces the probability of accepting worse solutions.

Working:

1. Begin with an initial signal-timing plan.
2. Generate a neighboring plan.
3. Accept it if it improves the objective.
4. Sometimes accept a worse plan based on a temperature parameter.
5. Gradually reduce the temperature and repeat.

Advantages:

- Can escape local optima.
- Can explore plateaus and alternative routes through the search space.
- Requires relatively little memory.

B. Genetic Algorithm (GA)

A Genetic Algorithm uses a population of possible solutions and improves them through selection, crossover, and mutation.

Working:

1. Generate multiple signal-timing plans.
2. Evaluate each plan using the objective function.
3. Select better-performing plans.
4. Combine plans through crossover.
5. Apply mutation to introduce new timing combinations.

6. Repeat until a satisfactory solution is obtained.

Advantages:

- Explores multiple solutions simultaneously.
- Can discover coordinated signal timings across intersections.
- Handles large, complex optimization problems.
- Mutation and population diversity help reduce premature convergence.

5. Recommend the best approach with justification

Recommendation: Genetic Algorithm with real-time traffic data and periodic optimization.

A Genetic Algorithm is a strong choice for a citywide problem because it evaluates multiple timing plans simultaneously and can optimize coordination across many intersections.

Criterion	**Hill Climbing**	**Simulated Annealing**	**Genetic Algorithm**
Escapes local optima	Poor	Good	Good, with sufficient diversity
Search approach	Single solution	Single solution	Population of solutions
Large search space	Limited	Suitable	Well suited
Real-time adaptability	Simple but may get stuck	Requires careful temperature tuning	Can periodically evolve solutions
Computational cost	Low	Moderate	Often higher

Implementation strategy:

- Collect live traffic data from cameras and sensors.
- Use a Genetic Algorithm to optimize signal-timing plans.
- Apply safety constraints, minimum and maximum green times, and pedestrian crossing requirements.
- Update signals periodically or when traffic conditions change significantly.
- Use a fast local controller to handle immediate adjustments while the Genetic Algorithm performs broader optimization.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing

capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

ANSWER:

Scenario 3: Online Search Agent for an Autonomous Mars Rover

1. Why does this problem require an online search agent instead of offline search?

An online search agent makes decisions while exploring the environment and learns about the surroundings through its sensors. An offline search agent plans using a known model of the environment before taking action.

The Mars rover does not have a complete map of the terrain. It must discover obstacles, identify safe paths, and collect samples while moving.

Advantages of an online search agent:

- Makes decisions using real-time sensor information.
- Discovers unknown obstacles while exploring.
- Updates its route whenever new information becomes available.
- Supports safe navigation in unfamiliar terrain.

Example: If the rover detects a crater ahead, it can update its map and choose another route instead of continuing toward the hazard.

2. Challenges of partial observability and dynamic environments

Partial observability: The rover cannot observe the entire terrain at once because its sensors have limited range and accuracy.

Challenges include:

- Hidden craters and rocks may remain undetected until the rover approaches them.

- Sensor noise may cause incorrect obstacle detection.
- Limited information makes route planning difficult.

Dynamic environments: Terrain conditions or the rover's operating conditions may change during exploration.

Challenges include:

- Dust may reduce visibility.
- Loose soil may affect movement.
- Unexpected hazards may block the planned route.
- Communication delays with Earth may prevent immediate human intervention.

Therefore, the rover must continuously observe, update its knowledge, and make safe decisions.

3. How does the rover build and update its internal model?

The rover maintains an internal model, such as a map or grid, representing known obstacles, safe areas, and unexplored regions.

Working process:

1. Sense: Collect information using cameras, radar, and other available sensors.
2. Interpret: Identify rocks, craters, slopes, and possible sample locations.
3. Update the map: Mark areas as safe, hazardous, or unknown.
4. Plan: Select a path toward the target using the current map.
5. Act: Move toward the next waypoint or collect a sample.
6. Replan: Update the route when new obstacles or unexpected conditions are detected.

Example: Initially, a region may be marked as unknown. After scanning it, the rover identifies a large rock and marks that location as hazardous. It then calculates an alternative path.

4. Compare online search with uninformed and informed search strategies

Feature	**Uninformed Search**	**Informed Search**	**Online Search**
Knowledge required	Uses available problem structure without a heuristic	Uses a heuristic to guide the search	Learns about the environment during exploration
Examples	BFS, DFS	A*, Greedy Best-First Search	Online DFS, online replanning methods
Complete map	Depends on the problem	Usually assumes a	No complete map

required	representation	sufficiently known search graph	required beforehand
Adapts to new observations	Not inherently	Not inherently	Yes
Suitable for the Mars rover	Limited in unknown terrain	Useful when a map and heuristic are available	Highly suitable

Uninformed and informed search describe how an algorithm selects nodes. Online search describes how an agent searches and acts when the environment is not fully known in advance. These approaches can also be combined.

5. Recommend the most suitable approach

Recommendation: Online search with heuristic-guided planning, such as A-based replanning, and safety checks.*

This approach is suitable because the rover must explore unknown terrain, use limited sensor information, and adapt its movements as new hazards are discovered.

Justification:

- Uncertainty: The rover updates its map using new sensor observations.
- Adaptability: It recalculates its route when an obstacle is detected.
- Efficiency: A heuristic guides the rover toward sample locations without exploring every possible path.
- Safety: Hazard detection, safe-distance constraints, and movement validation help prevent dangerous actions.
- Limited communication: The rover can make local navigation decisions without waiting for instructions from Earth.

A practical system would use sensors to update a local map, a heuristic planner to select promising routes, and a safety controller to reject unsafe movements. If the terrain changes, the rover replans using its updated information.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

ANSWER:

Scenario 4: AI-Based University Exam Timetable Generation Using CSP

1. Formulate the problem as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) is an AI problem consisting of variables, domains, and constraints. The goal is to assign a valid value to every variable without violating any constraints.

In this scenario, the AI system must assign examination dates, time slots, and exam halls to university courses while satisfying all academic and administrative requirements.

A. Variables

Variables represent the exams that need to be scheduled.

For example:

- E_1 = Java examination
- E_2 = Python examination
- E_3 = Mathematics examination
- E_4 = Computer Networks examination

Each exam variable can be assigned a time slot and an examination hall.

B. Domains

A domain is the set of possible values that can be assigned to a variable.

For example, the domain of the Java examination may contain:

- Monday, 9:00 AM — Hall A
- Monday, 1:00 PM — Hall B
- Tuesday, 9:00 AM — Hall A

The system removes options that are unavailable because of faculty schedules, hall capacity, or other restrictions.

C. Constraints

Constraints are rules that must be satisfied by the timetable.

Constraint	Explanation	Example
No overlapping exams	A student cannot take two exams at the same time.	Java and Python cannot overlap if students take both.
Limited exam halls	The number of exams scheduled simultaneously cannot exceed available hall capacity.	Only three halls are available.
Subject ordering	Certain subjects must be scheduled before others.	Exam A must occur before Exam B.
Faculty availability	Exams requiring a particular faculty member must fit their availability.	A faculty member cannot supervise two exams simultaneously.
Special accommodations	Students requiring special facilities must be assigned suitable rooms or time arrangements.	A student using a wheelchair needs an accessible hall.

The objective is to find a timetable that satisfies all hard constraints. Soft constraints, such as avoiding consecutive exams, can be optimized after feasibility is achieved.

2. Explain how backtracking search works

Backtracking search assigns values to variables one at a time. If an assignment violates a constraint, the algorithm goes back and tries another value.

Steps:

1. Select an unscheduled exam.
2. Choose a possible time slot and hall.
3. Check whether the assignment satisfies all constraints.
4. If valid, continue with the next exam.
5. If no valid assignment exists, return to the previous exam and try another option.
6. Repeat until all exams are scheduled or the system determines that no solution exists under the current constraints.

Example:

Suppose Java and Python share some students. If Java is scheduled on Monday at 9:00 AM, Python cannot be assigned to the same time slot. The system tries another slot for Python.

If all Python slots conflict with other constraints, the system backtracks and changes the Java assignment.

Advantage: Backtracking avoids continuing with assignments that already violate constraints.

Limitation: It may explore a large number of combinations when there are many courses and students.

3. Explain how constraint propagation improves efficiency

Constraint propagation reduces the possible values in variable domains whenever an assignment is made.

Forward checking

Forward checking checks the domains of unassigned variables after each assignment. It removes values that would conflict with the current assignment.

Example:

Suppose Java is assigned to Monday at 9:00 AM. Python shares students with Java, so the system removes Monday at 9:00 AM from Python's domain.

If Python has no remaining valid time slots, the system immediately backtracks instead of continuing with an impossible timetable.

Benefits:

- Detects conflicts early.
- Reduces the number of assignments explored.
- Saves computation time.
- Improves the efficiency of backtracking search.

Other useful techniques include:

- MRV (Minimum Remaining Values): Select the exam with the fewest available options first.
- Degree heuristic: Select the exam that constrains the largest number of other exams.
- Least Constraining Value: Choose the time slot that leaves the most options for other exams.

Combining these techniques can significantly improve the performance of a CSP solver.

4. Real-world challenges and the best strategy for scalability

Challenges

- Large search space: Hundreds of courses and many time slots create numerous combinations.
- Student conflicts: Courses may share students across different departments.
- Limited resources: Exam halls have capacity limits and specific facilities.
- Changing requirements: Faculty availability and exam dates may change.

- Special accommodations: Accessibility and individual student requirements must be considered.
- Conflicting constraints: Some combinations of requirements may make a valid timetable impossible.

Recommended strategy

Use backtracking search with constraint propagation and intelligent variable-ordering heuristics.

Technique	**Purpose**
Backtracking	Finds a valid assignment and reverses conflicting choices.
Forward checking	Removes invalid options early.
MRV heuristic	Schedules the most constrained exams first.
Degree heuristic	Prioritizes exams affecting many other exams.
Least Constraining Value	Preserves more options for remaining exams.
Incremental rescheduling	Updates only affected exams when changes occur.

For very large universities, the system can also divide the problem into manageable groups, such as departments or exam periods, while coordinating shared students and resources across groups.

Q5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

ANSWER:

Scenario 5: Strategic Game AI for Real-Time Decision Making (Minimax and Alpha-Beta Pruning)

1. Explain how the Minimax algorithm models this problem

The Minimax algorithm is an adversarial search algorithm used in games where two players compete against each other. It assumes that both players choose moves to achieve their objectives.

In this scenario, the AI tries to maximize its chances of winning, while the human opponent tries to minimize the AI's advantage.

Working of Minimax:

1. Represent the current game situation as the root node of a game tree.
2. Generate all possible moves for the AI.
3. Predict the opponent's possible responses.
4. Assign utility values to terminal game states.
5. At MAX nodes, select the highest value.
6. At MIN nodes, select the lowest value.
7. Choose the move that provides the best outcome against the opponent's possible responses.

Example: If the AI has three possible moves with minimax values of 5, 3, and 8, it selects the move with value 8, assuming the opponent's responses have already been considered.

2. Analyze the computational challenges of large game trees

A game tree grows rapidly as the number of possible moves and search depth increase.

The time complexity of Minimax is:

O(b^d)

Where:

- \(b\) = Average branching factor (number of possible moves per state).
- \(d\) = Search depth.

Major challenges:

- Large branching factor: Each game state may contain many possible actions.
- Increasing depth: Looking further ahead increases the number of states exponentially.
- Strict time limits: The AI may not finish searching before it must act.
- Complex game states: Resources, units, terrain, and enemy strategies make evaluation difficult.

- Real-time changes: The game state may change while the AI is planning.

Therefore, searching every possible move is often impractical in a real-time strategy game.

3. Explain how Alpha-Beta pruning improves efficiency

Alpha-Beta pruning improves Minimax by eliminating branches that cannot influence the final decision. It produces the same result as Minimax when both algorithms search to the same depth with the same evaluation assumptions.

- Alpha (α): The best value MAX can guarantee so far.
- Beta (β): The best value MIN can guarantee so far.
- Pruning condition: Stop exploring a branch when \(\alpha \geq \beta\).

Example

Consider a MAX node with two MIN children.

- The first MIN child has leaf values 3 and 5, so its value is 3. MAX now has a guaranteed value of at least 3, so alpha = 3.
- The second MIN child has a first leaf value of 2. Since MIN can already choose 2, this branch cannot produce a value better than 3 for MAX.
- The remaining children of the second MIN node can be skipped because they cannot change MAX's final decision.

Thus, Alpha-Beta pruning avoids unnecessary evaluations while preserving the Minimax result.

Benefits:

- Reduces the number of game states evaluated.
- Allows deeper searches within the same time limit.
- Improves decision-making efficiency.
- Does not change the final Minimax decision at the searched depth.

The worst-case time complexity remains O(b^d)but with ideal move ordering, the best-case complexity improves to approximately:

$$O\left(b^{d/2}\right)$$

This can allow the AI to search roughly twice as deep for a similar computational budget under favorable conditions.

4. Design an evaluation function and justify the factors considered

When the AI cannot search all the way to the end of a game, an evaluation function estimates how favorable a game state is for the AI.

A possible evaluation function for a real-time strategy game is:

$$E(s) = 10M + 5R + 8U + 6T + 7P - 9D$$

Where:

- M = Map control advantage.
- R = Resource advantage.
- U = Military unit strength advantage.
- T = Technological advantage.
- P = Progress toward the game objective.
- D = Damage or losses suffered relative to the opponent.

The weights are illustrative and should be tuned for the specific game.

Why these factors matter:

Factor	Reason
Map control	Gives access to strategic locations and resources.
Resources	Supports production, upgrades, and future actions.
Unit strength	Estimates military power and combat capability.
Technology	Represents advantages from research and upgrades.
Objective progress	Rewards actions that move the AI closer to victory.
Damage and losses	Penalizes costly engagements and weakened defenses.

The evaluation function should also recognize immediate winning or losing states using very large positive or negative scores. This ensures that winning the game is more important than gaining resources or territory.

5. Recommend a strategy for balancing optimality and real-time performance

Recommendation: Minimax with Alpha-Beta pruning, iterative deepening, and a carefully designed evaluation function.

This combination provides strong decisions while controlling computational cost.

Technique	Purpose
Minimax	Considers the opponent's possible responses.
Alpha-Beta pruning	Eliminates branches that cannot affect the decision.
Iterative	Searches depth 1, then 2, then 3, and so on, retaining the best completed

deepening	result.
Move ordering	Evaluates promising moves first to improve pruning.
Evaluation function	Estimates non-terminal game states.
Time limit	Ensures the AI can act within the available decision window.

Implementation strategy:

1. Start with a shallow search to obtain a valid move quickly.
2. Use iterative deepening to explore deeper levels while time remains.
3. Apply Alpha-Beta pruning with promising moves evaluated first.
4. Use a time-aware evaluation function to rank game states.
5. Stop when the time budget is nearly exhausted and return the best move from the last completed search.

For a complex real-time strategy game with simultaneous actions, hidden information, or thousands of possible moves, pure Minimax may not be sufficient. The company could combine this approach with beam search, hierarchical planning, or domain-specific rules to reduce the search space.